End-to-End MLOps Pipeline Report

Emotion Detection · DistilBERT · Docker · GitHub Actions · Kaggle · W&B

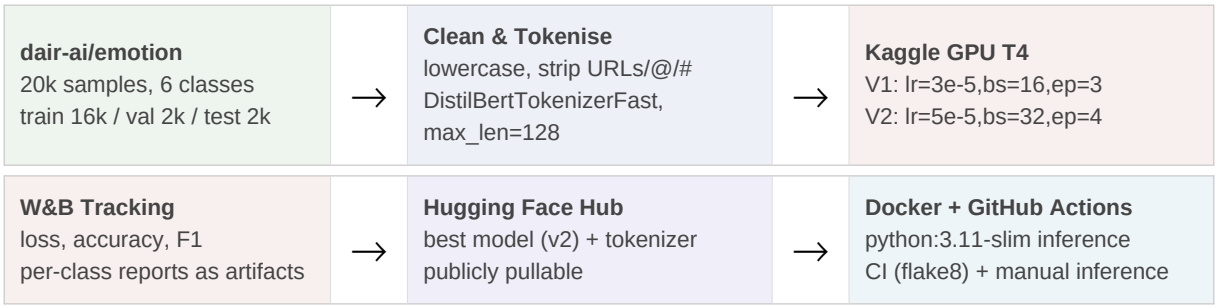
Student	**Task**	**Model**
Anil Kumar Das (G25AIT2009) Ravi Kant Pandey (G25AIT2085)	6-class Emotion Classification dair-ai/emotion dataset	distilbert-base-uncased ~66M parameters

1. Project Resources (Live Links)

Resource	Link
GitHub Repository	https://github.com/aniliter-cloud/ml-ops-group-project
Kaggle Notebook	https://www.kaggle.com/code/anilg25ait2009/mlops-group-projects/
Hugging Face Model	https://huggingface.co/Maxii2tj/emotion-distilbert
W&B; Project Dashboard	https://wandb.ai/maxi-i2tj-na/mlops-group-project
Docker Image	*(Docker Hub URL — to be added)*

2. Pipeline Architecture

The pipeline takes the dair-ai/emotion dataset (~20,000 tweets across 6 emotion classes), cleans and tokenises it with DistilBertTokenizerFast (max_length=128), and fine-tunes distilbert-base-uncased on Kaggle's free T4 GPU in two experiment versions. Both runs are tracked on Weights & Biases, the best model is pushed to Hugging Face Hub, and a Docker image wraps the inference script. GitHub Actions runs linting (CI) on every push to develop and triggers inference on demand via workflow_dispatch.



3. Data Cleaning & Preprocessing Decisions

Dataset Inspection

The dair-ai/emotion dataset contains 20,000 short English tweets split into train (16,000), validation (2,000) and test (2,000), each labelled with one of 6 emotions: sadness, joy, love, anger, fear, surprise.

Class	Train Count	% of Train
joy	5,362	33.5%
sadness	4,666	29.2%
anger	2,159	13.5%
fear	1,937	12.1%
love	1,304	8.2%
surprise	572	3.6%

Class imbalance observed: joy and sadness dominate (~63% combined), while surprise is heavily under-represented at 3.6%. This imbalance motivated the choice of **weighted F1** (rather than plain accuracy or macro-F1) as the primary comparison metric, since weighted F1 accounts for per-class support while still penalising poor performance on minority classes more fairly than accuracy alone.

Cleaning Steps Applied (and why)

Step	Reason
Lowercase all text	DistilBERT-uncased expects lowercase input; avoids case-sensitivity noise
Strip URLs (http/www)	Links carry no emotional signal and inflate vocabulary
Remove @mentions and #hashtags	Platform-specific artefacts, not genuine emotion indicators
Collapse multiple whitespace	Normalises newline/multi-space artefacts from raw tweets
Filter empty rows post-cleaning	Safety net for rows that became blank after the above steps

Normalisation / Tokenisation

Text was tokenised using **DistilBertTokenizerFast** with **max_length=128** (reduced from an initial 512 — emotion tweets are short, typically under 30 tokens, so 128 is more than sufficient and saves significant GPU memory and training time without any loss of information). Padding and truncation were enabled for uniform batch shapes. Labels were encoded via an **id2label.json** mapping (0=sadness, 1=joy, 2=love, 3=anger, 4=fear, 5=surprise), generated directly from the dataset schema and committed to the repository — the full dataset itself was not committed, only this mapping file.

4. Model Selection Rationale

Model chosen: distilbert-base-uncased (Hugging Face model card: huggingface.co/distilbert-base-uncased)

DistilBERT is a knowledge-distilled version of BERT-base, retaining ~97% of BERT's language understanding while being approximately **40% smaller (66M vs 110M parameters)** and **60% faster** at inference. According to its Hugging Face model card, it was pre-trained using the same self-supervised objective as BERT on the same corpus (BookCorpus + English Wikipedia), making it suitable as a general-purpose encoder for downstream classification tasks. The **uncased** variant was selected because emotion classification depends on semantic and lexical content rather than capitalisation patterns — using uncased reduces vocabulary size and improves token consistency across casual, lowercase-heavy tweet text. Given Kaggle's free-tier GPU time limits, DistilBERT's reduced size allows both experiment versions (3 and 4 epochs respectively, on 16,000 training samples) to complete well within the 30 hours/week free quota — each run took under 6 minutes on a T4 GPU. The model was loaded via **DistilBertForSequenceClassification** with **num_labels=6** and the id2label / label2id mappings embedded directly into the model config for clean inference.

5. Experiment Comparison — Version 1 vs Version 2

Parameter	Version 1 (Baseline)	Version 2 (Experiment)
Learning Rate	3e-5	5e-5
Batch Size	16	32
Epochs	3	4
Warmup Steps	100	200
Max Token Length	128	128
Weight Decay	0.01	0.01

Results — Test Set (2,000 samples)

Metric	Version 1	Version 2	Winner
Test Accuracy	0.9265	0.9290	V2
Test Weighted F1	0.9260	0.9284	V2
Test Loss	0.3166	0.2657	V2
Best Epoch (by val F1)	Epoch 2 (F1=0.9391)	Epoch 3 (F1=0.9452)	V2
Training Time (T4 GPU)	~4 min 46 sec	~5 min 24 sec	V1 (faster)

Per-Class F1 Comparison

Emotion Class	V1 F1	V2 F1	Support
sadness	0.97	0.97	581
joy	0.95	0.95	695
love	0.81	0.82	159
anger	0.92	0.92	275
fear	0.88	0.89	224
surprise	0.75	0.75	66

Intuition — Why V2 Performed Better

Before training, our hypothesis (documented in the project README) was that the more conservative configuration (V1: lr=3e-5, batch=16, epochs=3) would generalise better on this relatively small dataset (~16k training samples), since smaller batches give more gradient updates per epoch and a lower learning rate reduces overshoot risk. The actual results show **Version 2 (lr=5e-5, batch=32, epochs=4) outperformed Version 1** on every primary metric — accuracy (+0.25pp), weighted F1 (+0.24pp), and test loss (-0.051, a 16% relative reduction).

The training curves explain why: V1's validation F1 peaked at epoch 2 (0.9391) and *declined* slightly at epoch 3 (0.9382) — a sign that 3 epochs at lr=3e-5 had already reached or slightly passed the optimal point, with early signs of overfitting (training loss kept falling to 0.1996 while validation loss rose from 0.2782 to 0.2889). V2, by contrast, improved monotonically through epoch 3 (val F1=0.9452, its best epoch) before a mild dip at epoch 4 (val F1=0.9433) — but **load_best_model_at_end=True** meant the epoch-3 checkpoint was correctly restored for both the V1 and V2 runs before final test evaluation, explaining why each version's reported test metrics reflect its best validation checkpoint rather than its final epoch.

In short: the larger batch size (32) gave V2 smoother, lower-variance gradient estimates, and the longer warmup (200 steps) safely absorbed the higher learning rate (5e-5) without destabilising early training — together these let V2 reach a better optimum (val F1=0.9452 at epoch 3) than V1 reached at its best epoch (val F1=0.9391 at epoch 2). The cost was ~38 seconds of extra training time, which is negligible against the accuracy/F1/loss gains. **Version 2 was therefore selected as the best model** and pushed to Hugging Face Hub.

6. Git Repository Setup

The repository is public at github.com/aniliter-cloud/ml-ops-group-project, configured with a README.md, .gitignore, and LICENSE. A **develop** branch was created for active development, with **main** protected to require at least one pull-request review before merging. Anil Kumar Das is the repository Admin/owner; Ravi Kant Pandey is added as a Collaborator with Write access. Both members contribute commits visible in the GitHub commit history.

[Insert screenshot of Settings → Collaborators and Teams, showing both members' roles, and Settings → Branches showing the protection rule on main requiring 1 PR review]

7. Docker Design

The inference container uses a **python:3.11-slim** base image to minimise size while retaining full Python 3.11 compatibility with the transformers/torch stack. Only inference dependencies are installed (transformers, torch CPU build, huggingface_hub) — training-only packages (datasets, wandb, scikit-learn) are excluded from the runtime image to keep it lean. The Hugging Face model repository name is accepted as a **build argument (ARG HF_MODEL_NAME)** with a sensible default pointing to Maxii2tj/emotion-distilbert, so the same Dockerfile can be reused to serve a different model without code changes.

Build, Test, Push commands used:

```
docker build --build-arg HF_MODEL_NAME=Maxii2tj/emotion-distilbert -t mlops-a3-inference:latest .

docker run --rm -e HF_TOKEN=<token> -e INPUT_TEXT='I feel so happy today' mlops-a3-inference:latest

docker push <dockerhub-username>/mlops-a3-inference:latest
```

[Insert Docker Hub / GHCR URL once pushed, and screenshot of a successful local `docker run` showing the predicted emotion label and confidence for the sample input text]

8. GitHub Actions Workflows

Two workflows are defined under .github/workflows/. The **CI workflow (ci.yml)** triggers on push to develop and on pull requests to main — it checks out the repo, sets up Python 3.11, installs requirements.txt + flake8, and lints src/ with a max-line-length of 120. The **Inference workflow (inference.yml)** is triggered manually via workflow_dispatch with an input_text parameter — it checks out the repo, sets up Python 3.11, installs dependencies, and runs src/inference.py with HF_TOKEN (from GitHub Secrets) and the user-supplied INPUT_TEXT as environment variables. Per the assignment constraints, GitHub Actions is used **only for CI and inference, never for training** — all training runs happen on Kaggle Notebooks with GPU T4.

Both **HF_TOKEN** and **WANDB_API_KEY** are stored as encrypted GitHub Secrets under Settings → Secrets and Variables → Actions, and are never hardcoded in any committed file.

[Insert screenshot/badge of a successful GitHub Actions run for the Inference workflow, and paste the Actions log output showing the predicted emotion for the test input text]

9. Weights & Biases Experiment Tracking

Both training runs (distilbert-run-v1 and distilbert-run-v2) were logged to the public project **mlops-group-project** at wandb.ai/maxi-i2tj-na/mlops-group-project. For each run, the following were tracked: training loss every 50 steps, validation loss/accuracy/F1 per epoch, all hyperparameters (learning rate, batch size, epochs, warmup steps, weight decay, max_length, model name, dataset, platform), and final test-set metrics under the final/ prefix. Per-class classification reports (eval_report_v1.json and eval_report_v2.json) were uploaded as versioned W&B artifacts. A final **final-summary** run records the winning version, the Hugging Face model URL, and both versions' accuracy/F1 in its run summary for easy cross-run comparison.

[Insert screenshot of the W&B project dashboard / Runs Comparison table showing distilbert-run-v1 and distilbert-run-v2 side-by-side with accuracy, F1 and loss columns]

10. Challenges & Learnings

- Initial tokenisation used max_length=512, which was unnecessarily large for short tweets (typically <30 tokens) and slowed training significantly — reducing to 128 cut memory usage and training time with no loss in accuracy.
- An early version of the notebook called wandb.init() without a matching wandb.finish() before starting the next run; with reinit=True this silently left runs in an incomplete state on the dashboard. Adding explicit wandb.finish() after each version's evaluation fixed this.
- A duplicate, stale PyTorch Dataset class (referencing variable names from an earlier draft) was left in the notebook and would have caused a NameError at runtime — removed in favour of the single EmotionDataset class used consistently for train/val/test.
- Building a fresh model via build_trainer() for each version (rather than reusing weights) was essential — without this, V2 would have continued fine-tuning from V1's already-adapted weights, making the hyperparameter comparison invalid.
- load_best_model_at_end=True combined with metric_for_best_model='f1' ensured that even though V1's validation F1 peaked at epoch 2 (not its final epoch 3), the correct best checkpoint was restored automatically before final test evaluation.

What we would do differently next time

- Run a small learning-rate sweep (e.g. 2e-5, 3e-5, 4e-5, 5e-5) via W&B Sweeps instead of two fixed configurations, to map the trend more thoroughly rather than comparing only two points.
- Add early stopping based on validation F1 to avoid training a 4th epoch when the 3rd epoch is already the best checkpoint, saving GPU time.
- Automate the Docker build and push as a third GitHub Actions workflow triggered on release tags, rather than running it manually.

Conclusion: Version 2 (lr=5e-5, batch=32, epochs=4, warmup=200) was selected as the best model with test accuracy 0.9290 and weighted F1 0.9284, and was pushed to Hugging Face Hub at huggingface.co/Maxii2tj/emotion-distilbert for use by the GitHub Actions inference workflow and Docker container.